

Aufgabenblatt 2

Stapel und Warteschlangen

Wichtige Ankündigungen

- Das Vorlesungsmaterial, die Übungsblätter und die Vorlagen für die Hausaufgaben finden Sie unter <https://git.tu-berlin.de/algodat-sole25/material.git>.
- Alle Hausaufgaben sind in Einzelarbeit zu erledigen. Kopieren Sie niemals Code und geben Sie Code in keiner Form weiter.
- Wenn Ihre Abgabe nicht im richtigen Ordner liegt, nicht kompiliert, unerlaubte packages oder imports enthält oder zu spät abgegeben wird, gibt es **0 Punkte** auf diese Abgabe.
- Es gibt drei Aufgabentypen:
 - **Tutorium** → Besprechung im Tutorium
 - **Hausaufgabe** → Eigenarbeit mit Hilfe in Rechnerübungen
 - **Klausurvorbereitung** → Optionale Klausurübung

Abgabe (bis 26.05.2025 23:59 Uhr)

Die folgenden Dateien werden für die Bepunktung berücksichtigt:

Geforderte Dateien:

Blatt02/src/Dec2Bin.java	Aufgaben 3.1 und 3.2
Blatt02/src/Bettelman.java	Aufgabe 4.2

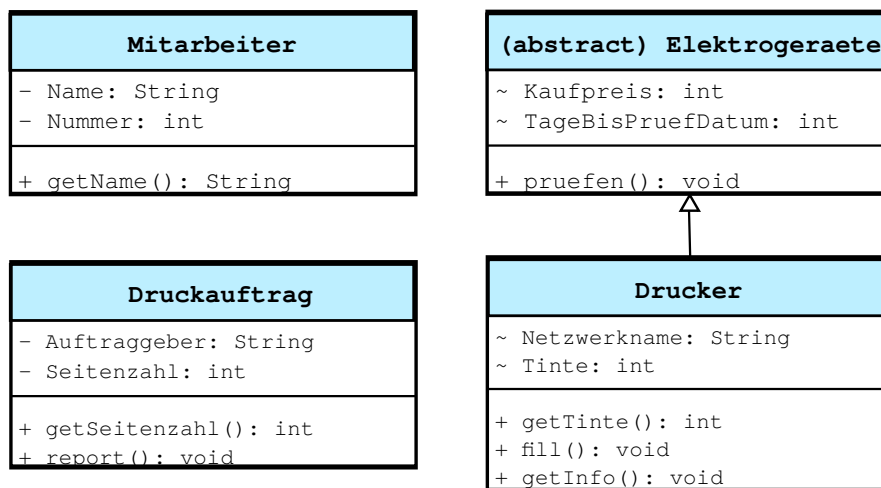
Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

Aufgabe 1: Das Drucker-Dilemma – Warteschlangen (Tutorium)

1.1 Wiederholung Vererbung

Implementieren Sie die folgenden Klassen:

(abstract) Elektrogeraete, Drucker, Mitarbeiter und Druckauftrag, wobei Sie die Attribute und Methoden der Klassen diesen UML Diagrammen entnehmen können:



wobei die Klasse Drucker von der Klasse Elektrogeraete erbt.

Die Methoden sollen folgendes implementieren:

- `report()`: gibt Auftraggeber und Seitenzahl auf der Kommandozeile aus.
- Der default für `TageBisPruefDatum` = 365
- `pruefen()`: setzt `TageBisPruefDatum` auf 365
- Der default für `Drucker.Tinte` = 100
- `fill()`: setzt Tinte auf 100
- `getInfo()`: gibt Druckernamen, Tinte, und `TageBisPruefDatum` auf der Kommandozeile aus

1.2 Queues (und Stacks) in Java

Definieren Sie Stapel und Warteschlangen. Erstellen Sie eine `main`-Methode in der Klasse `Druckauftrag` und legen Sie zunächst einen Stapel und dann eine Warteschlange von Druckaufträgen in Java an. Beachten Sie, dass Warteschlangen in Java als Interface implementiert sind und Stapel als Klasse und in beiden Generics benutzt werden.

1.3 Drucken

Nutzen Sie die vorgegebenen Klassen `Firma`, `Drucker` und Ihre in Aufgabe 1.1 erstellten Klassen `Elektrogeraete`, `Mitarbeiter` und `Druckauftrag`. Implementieren Sie in der Klasse `Drucker` die Methoden `fill`, `getTinte` und `drucken`.

`drucken()` soll alle Aufträge aus der Queue Druckaufträge bearbeiten. Beachten Sie, dass nur gedruckt werden kann, wenn genügend Tinte vorhanden ist und `TageBisPruefDatum` > 0 ist. Ansonsten muss die Tinte aufgefüllt oder das Elektrogerät geprüft werden. Eine Seite braucht beim Drucken eine Einheit Tinte. Der Einfachheit halber können nur Druckaufträge angelegt werden, die zwischen 0 und 100 Seiten groß sind. Diese Funktionalität ist schon über

`Firma.Druckauftragssteuerung()` implementiert. Das bedeutet, dass sie immer nur zu Beginn des Druckens prüfen müssen, ob genug Tinte vorhanden ist und diese bei Bedarf auffüllen müssen.

Um eine Wartezeit beim Drucken zu simulieren, können sie die Methode `warten` benutzen. Testen können sie Ihre Implementierung, in dem sie die `main` in der Klasse `Firma` ausführen.

Lösung

Siehe Code im Repo

Aufgabe 2: Laufzeit kleiner Programmabschnitte (Tutorium)

Bestimmen Sie die Wachstumsordnung der Laufzeit von den in Listing 1 angegebenen Funktionen `f1` bis `f6`.

Listing 1: Java Snippets zur Laufzeitbestimmung

```
public static int f1(int N) {
    int x = 0;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            x++;
    return x;
}
```

(a) Funktion f1

```
public static int f2(int N) {
    int x = 0;
    for (int i = 0; i < N/2; i++)
        for (int j = 0; j < i; j++)
            x++;
    return x;
}
```

(b) Funktion f2

```
public static int f3(int N) {
    if (N == 0)
        return 1;
    else
        return 2 * f3(N - 1) + 1;
}
```

(c) Funktion f3

```
public static int f4(int N) {
    int x = 0;
    for (int i = 1; i < N; i *= 2)
        x += f3(i); // f3!
    return x;
}
```

(e) Funktion f4

```
public static int f5(int N) {
    if (N <= 1)
        return N+1;
    else
        return f5(N-2) / f5(N-1);
}
```

(d) Funktion f5

```
public static int f6(int N) {
    if (N == 0)
        return 1;
    return 2 * f6(N/2);
}
```

(f) Funktion f6

Es gibt ein Videotutorium zu dieser Aufgabe. Die darin vorgestellte Lösung zu `f4` ist nicht scharf, für ein Beispiel, wie man für diese Funktion auch das richtige Θ bestimmt, können Sie die schwereren Beispiele im Skript durchgehen. Für die Klausur reichen die im Video vorgestellten Vorgehensweisen.

Lösung

- $f1(n) \in O(n^2)$ Berechnet N^2 . Siehe auch VL-Skript Seite 15: `f2()`
- $f2(n) \in O(n^2)$ Berechnet $\frac{N^2+2N}{8}$. Siehe auch VL-Skript Seite 16: `f4()`
- $f3(n) \in O(n)$ Berechnet $2^{n+1} - 1$. Siehe auch VL-Skript Seite 18: `r1()`
- $f4(n) \in O(n \cdot \log n)$, aber $\Theta(n)$. Siehe dazu VL-Skript Seite 17: `f6()`
- $f5(n) \in O(2^n)$. Siehe dazu VL-Skript Seite 19: `r3()`
- $f6(n) \in O(\log n)$. Siehe dazu VL-Skript Seite 19: `r2()`

Aufgabe 3: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Aufgabe 4: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Aufgabe 5: Stapel und Warteschlangen umkehren (Klausurvorbereitung)

Stapel und Warteschlangen funktionieren in Bezug auf die Abrufreihenfolge gegensätzlich. Daher kann man die Reihenfolge der Elemente in einem Stapel umkehren, indem man sie aus dem Stapel holt, in eine (leere) Warteschlange einfügt, sie dann aus der Warteschlange holt und wieder auf den (leeren) Stapel legt. Für Warteschlangen geht die Umkehrung entsprechend über einen Stapel.

Implementieren Sie in der Klasse `QueuesAndStacks` die statischen Methoden

```
public static <E> void reverse(Stack<E> stack)
public static <E> void reverse(Queue<E> queue)
```

Sie sollen die Reihenfolge der Elemente eines Stapels bzw. einer Warteschlange umkehren, indem sie das oben skizzierte Verfahren benutzen. Die Methoden benutzen den generischen Typ `E` für die Elemente der jeweiligen Datenstruktur. Testen Sie Ihre Methoden mit der vorbereiteten `main()` Methode. Sie benutzt die ebenfalls schon implementierten Methoden `stackToString()` und `queueToString()`. Schauen Sie auch, zu welcher Ausgabe die Benutzung der Methoden `Stack.toString()` und `Queue.toString()` führen. Sie werden implizit in den beiden auskommentierten Zeilen aufgerufen.

Lösung

Siehe Code im Repo

Was Sie nach diesem Blatt wissen sollten:

- was Objekte und Klassen sind und wie Vererbung funktioniert.
- wie man Konstruktoren schreibt, Objekte erzeugt und Methoden auf diesen Objekten aufruft.
- wie Sie die Laufzeit von Methoden bestimmen.
- was Collections in Java sind.
- wie Stacks und Queues funktionieren, was die Unterschiede sind und wie man sie nutzt.
- was die Schnittstellen `Comparable` (und `Comparator`, `Iterator` und `Iterable`) sind und wie man sie implementiert.
- wie Sie eine `toString()`-Methode schreiben und was das ist.
- wie Sie while-Schleifen verwenden.
- wie Sie eine `main`-Methode zu Testzwecken anpassen und ausführen.
- wie Sie Ihre geschriebenen Methoden selbst ausprobieren und überprüfen.